

What a web browser is

A web browser is an application that fetches resources (HTML, CSS, JavaScript, images, video, etc.) from networks or local storage, interprets them according to web standards, and presents an interactive document (a web page) to the user. It also exposes a security-constrained programming environment (Web APIs + JavaScript) so pages can be dynamic, connected, and app-like.

How a browser works (high level → deep dive)

1) Navigation and fetching

Input and URL handling

You type a URL, search query, or click a link. The browser parses the input, normalizes the URL, and consults history/bookmarks/omnibox suggestions.

If a Service Worker is registered for the scope, it can intercept the navigation and serve content from its cache or forward requests.

Name resolution and connection

DNS lookup (or DoH/DoT) resolves the hostname to IPs; the browser chooses a destination using connection pooling and racing strategies.

Establishes a secure connection: TLS handshake (or QUIC for HTTP/3), validates certificates, enforces HSTS, and applies mixed-content and upgrade rules.

HTTP request/response

Sends request headers (cookies, Accept, etc.; modern browsers reduce/“freeze” User-Agent details), supports redirects, authentication, compression, and content negotiation.

Receives a streaming response; caching rules (Cache-Control, ETag/If-None-Match, Last-Modified/If-Modified-Since) determine fresh vs. revalidated content.

2) Parsing and rendering pipeline

HTML tokenization → DOM

As HTML bytes stream in, an HTML tokenizer and parser build the Document Object Model (DOM). The parser runs a “preload scanner” to discover subresources early (CSS, JS, images, fonts).

CSS parsing → CSSOM

Stylesheets are fetched and parsed into a CSS Object Model (CSSOM). CSS can block rendering until it’s ready because style affects layout.

JavaScript execution

External and inline scripts execute via the JS engine; defer, async, and module semantics control when scripts run relative to parsing. JS can modify DOM/CSSOM.

Engines compile and optimize JS (parser → bytecode → tiered JITs), manage memory via garbage collection, and coordinate with the event loop.

Style, layout, paint, composite

Style calculation: cascade + inheritance + selectors resolve computed styles for each DOM node.

Layout (reflow): calculates geometry for the “box tree” (inline, block, flex, grid, etc.), respecting fonts, images, and intrinsic sizes.

Paint: draws backgrounds, borders, text, images on display lists.

Compositing: splits content into layers; the compositor threads schedule rasterization and GPU-accelerated compositing for smooth scrolling and animations (avoids main-thread jank when possible).

Incremental/interactive updates

The event loop (tasks, microtasks, rendering steps) interleaves network, script, and UI. DOM mutations trigger style/layout/paint invalidations as needed (browsers

minimize work with dirty bit sets and heuristics).

3) Runtime and platform features

Web APIs (security sandboxed)

DOM, Fetch/XHR, Streams, WebSocket/WebTransport, WebRTC, Canvas/WebGL/WebGPU, Media, Storage (Cookies, localStorage, sessionStorage, IndexedDB, Cache API), Intl, Sensors/Geolocation (permission-gated), and more.

Storage & offline

HTTP cookies; Storage partitioning and SameSite to reduce tracking; IndexedDB/Cache API for structured/offline data; Service Workers for offline/PWA behaviors and background sync.

Performance strategies

Preload/prefetch/prerender, connection reuse, HTTP/2 multiplexing, HTTP/3 with QUIC, speculative parsing, code-splitting, lazy loading, font subsetting, GPU acceleration.

Security model

Same-Origin Policy, CORS, CORB/COOP/COEP, CSP, sandbox iframes, site isolation (multi-process per site), permission prompts, anti-XSS/anti-clickjacking, mixed-content blocking, certificate pinning/HSTS preload lists.

Privacy

Tracking protection, third-party cookie restrictions/phase-out, fingerprinting mitigations, partitioned state (cookies/cache), private browsing modes.

Process and sandbox architecture

Modern browsers use a multi-process model: a privileged browser process, GPU process, network process, and isolated renderer processes per site (or site instance). OS sandboxes constrain what each process can do.

DevTools & updates

Built-in DevTools (Elements, Network, Performance, Memory, Sources, Application, Security). Auto-update channels (stable/beta/dev/canary) deliver frequent engine and security fixes.

4) Engines (core components)

Rendering engines: Blink (Chromium/Chrome/Edge/Opera), WebKit (Safari), Gecko (Firefox).

JavaScript engines: V8 (Chromium-based browsers), JavaScriptCore (Safari), SpiderMonkey (Firefox).

Timeline and history of the first popular browsers

1990–1991: WorldWideWeb (later “Nexus”)

Created by Tim Berners-Lee at CERN; the first browser/editor. Text-based with simple rendering, largely a NeXTSTEP app.

1992: ViolaWWW

Early graphical browser with scripting; influential but limited adoption.

1993: NCSA Mosaic

First widely popular graphical browser (Windows/Mac/UNIX). Introduced inline images with text—made the web accessible to non-technical users and catalyzed web growth.

1994–1997: Netscape Navigator

Founded by ex-Mosaic developers (Marc Andreessen/Jim Clark). Navigator dominated mid-1990s, introduced cookies, frames, and early JavaScript (LiveScript → JavaScript).

1995–2001: Microsoft Internet Explorer (IE)

Started with IE1 (based on Spyglass Mosaic), accelerated with IE3/IE4 (ActiveX, CSS1), and IE6’s long tail on Windows XP. The “browser wars” drove rapid (and often

proprietary) feature competition.

1995–2013: Opera

Innovative on performance and features (tabs, speed dial, mouse gestures). Used the Presto engine before switching to Chromium/Blink in 2013.

1996–2003: KHTML/Konqueror → WebKit

KDE's KHTML engine became the basis for Apple's WebKit (2002–2003). WebKit powered Safari and later inspired other engines.

2003: Apple Safari

Fast, standards-oriented browser for macOS (and later iOS). WebKit remains the core, with strong power efficiency on Apple hardware.

2002–2004: Phoenix/Firebird → Mozilla Firefox

A leaner successor to the Mozilla Suite, Firefox 1.0 (2004) reignited competition and standards adoption; introduced robust extensions and strong security messaging.

2008: Google Chrome

Introduced V8 (high-performance JS), multi-process sandboxing, fast release cadence, and later Blink (a 2013 fork of WebKit). Popularized automatic updates and rigorous benchmarking.

2015 → 2020: Microsoft Edge

EdgeHTML (2015) replaced IE as the default on Windows 10; in 2020 Microsoft rebuilt Edge on Chromium (Blink + V8), retaining Microsoft's UX/features while aligning with the Chromium ecosystem.

Mobile milestones

2007: Mobile Safari defined modern mobile web; Android Browser (WebKit-based) followed, then Chrome for Android (2012). Mobile pushed touch, responsive design, and power-efficiency focus.